B. Data Structures, Symmetries, and Neural Network Architectures

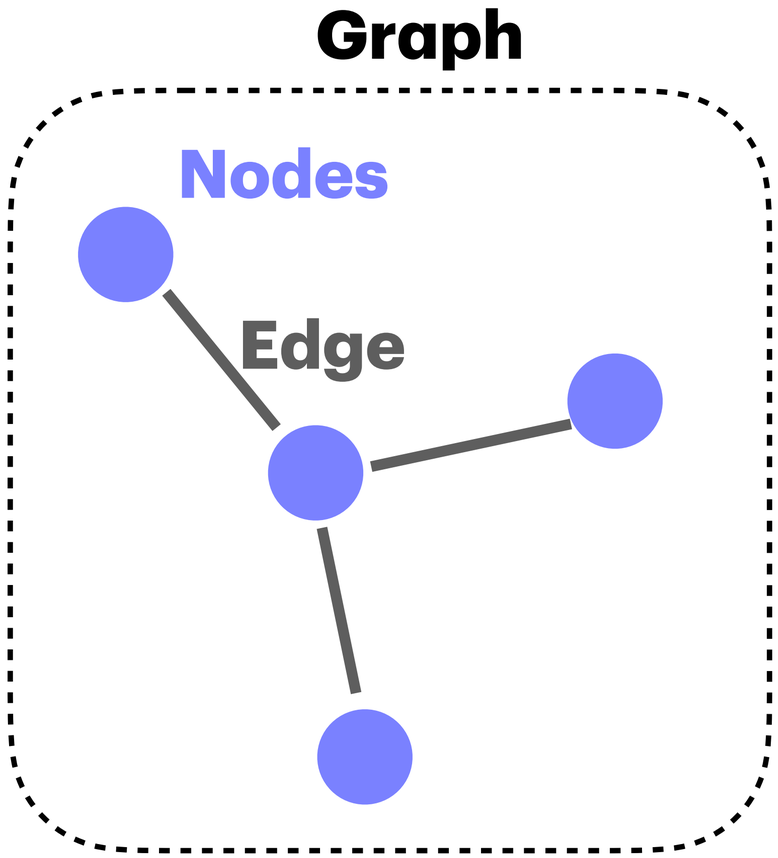
B.1 Tabular data: MLPs, residual connections, and autoencoders
B.2 Grid-structured data: CNNs
B.3 Sequential data: RNNs, LSTMs, and GRUs
B.4 Graph-structured data: GNNs and message passing
B.5 Set-structured data: Deep Sets

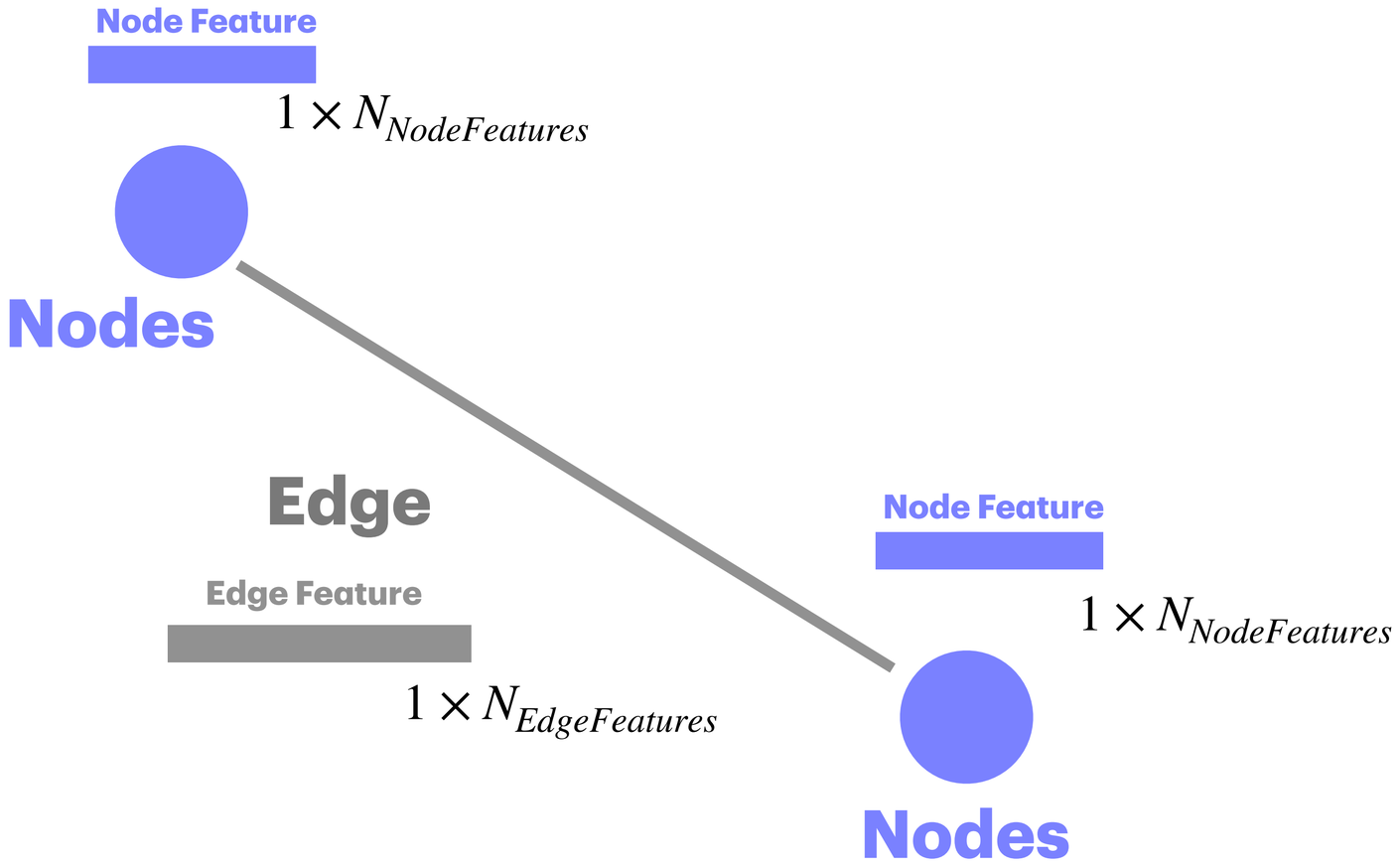
Tiam Heydari, PhD
Postdoctoral Fellow, UBC

Part I: Graphs

Graphs: Introduction

How we store information about nodes and edges?

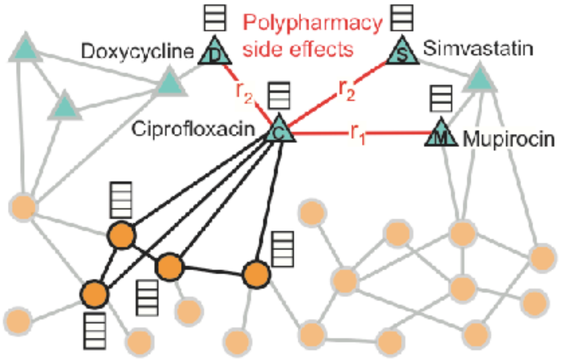


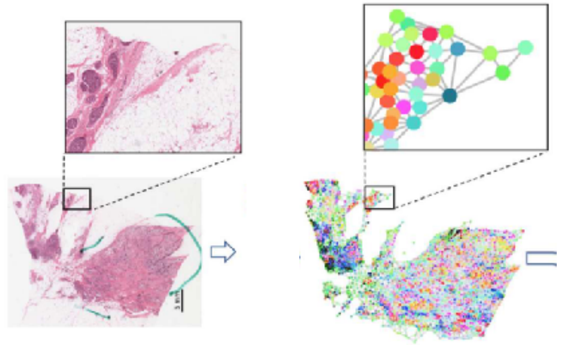


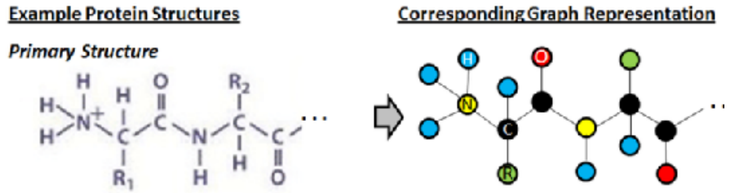
Graphs: Introduction

Some data are best naturally represented as graphs

Context	Node	Edges	Node features	Edge Features
Social Networks	Users	Friendship	Age, location, replies, interests, gender	—
Drug Repurposing	Drugs	Drug Interactions	Chemical fingerprints, molecular weight	binding affinity, side-effect profile of interaction
Pathology	Individual cells	Spatial proximity	Cell morphology, nuclear shape	Distance, spatial adjacency
Proteins	individual atoms	bonds	Polarity, charge, hydrophobicity, atom type	Bond type, 3D distance, interaction strength

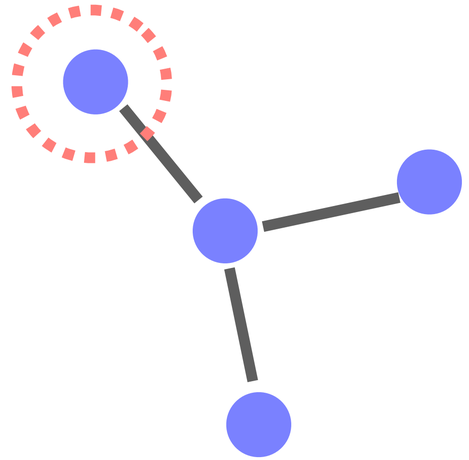






Graphs: Introduction

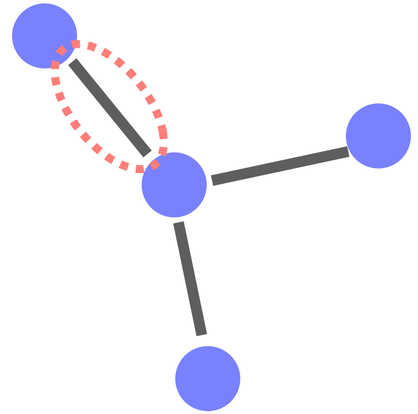
What type of predictive tasks we can do with graphs?



A- Node level tasks:

Task type	Output	Typical learning	Example(s)
Node classification	Label per node	Supervised	Cell type per cell (spatial graph); residue class in protein structure graph
Node regression	Value per node	Supervised	Sensor-level rainfall forecast in station graph
Node ranking / importance	Score/rank per node	Unsupervised or Supervised	Essential genes/proteins as a disease factor
Node clustering / community detection	Cluster/community id	Unsupervised	Disease modules; social communities

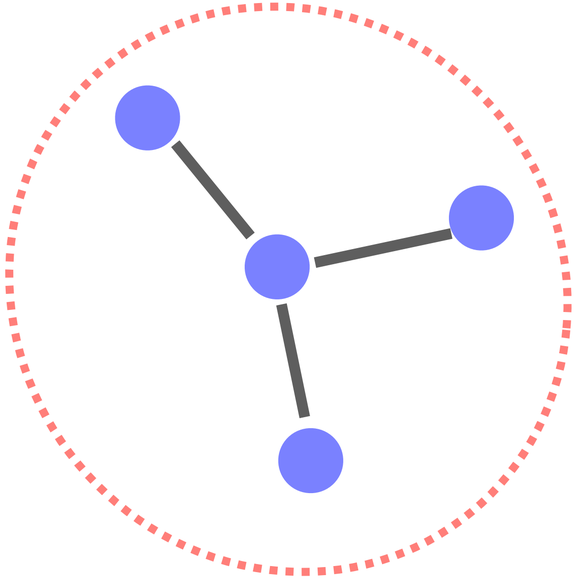
Graphs: Introduction



B- Edge level tasks:

Task type	Output	Typical learning	Example(s)
Link prediction	edge exists for node pair	Supervised or Self-supervised	Predict missing friendships; predict drug–target links
Edge classification	Relation label per edge	Supervised	Synergistic vs antagonistic drug pairs; relation type in knowledge graphs
Edge regression	Value per edge	Supervised	Interaction strength; binding affinity on drug–target edges
Temporal edge forecasting	Future edge/weight	Supervised	Future messages between users; evolving contacts in tissues

Graphs: Introduction



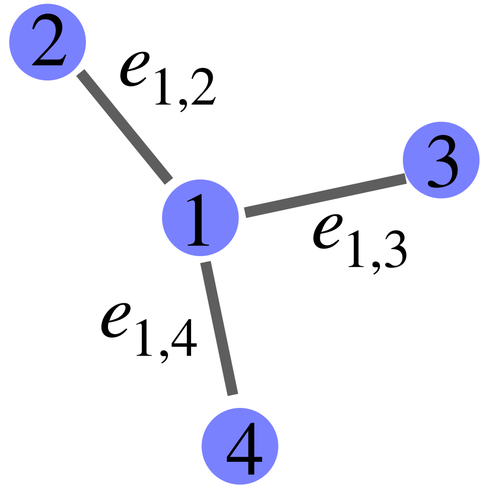
C- Graph level tasks:

Task type	Output	Typical learning	Example(s)
Graph classification	Label per graph	Supervised	Molecule toxic/non-toxic
Graph regression	Value per graph	Supervised	Solubility; conductivity
Graph similarity / retrieval	Nearest graphs / similarity score	Self-supervised or Supervised	Find similar molecules; find similar patients, find similar
Graph generation	New graph structure	Self-supervised, generative	De novo molecular design

Graphs: How we specify graphs mathematically

The **topology** of graph G, with a set of nodes V and edges E is specified as: **G=(V,E)**

$$V = \{1,2,3,4\}$$



- Two ways to specify the edges:

A) As a set of edges:

$$E = \{e_{1,2}, e_{1,3}, e_{1,4}\} \quad \Longleftrightarrow \quad \text{EdgeIndex} = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 3 & 4 \end{bmatrix}$$

B) Or via adjacency matrix **A**:

$$A_{ij} = \begin{cases} 1, & ;\text{if } \{i,j\} \in E \\ 0, & ;\text{otherwise} \end{cases}$$

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

Graphs: How we specify graphs mathematically

Node and Edge features, are specified as matrices

For one node

Node Feature

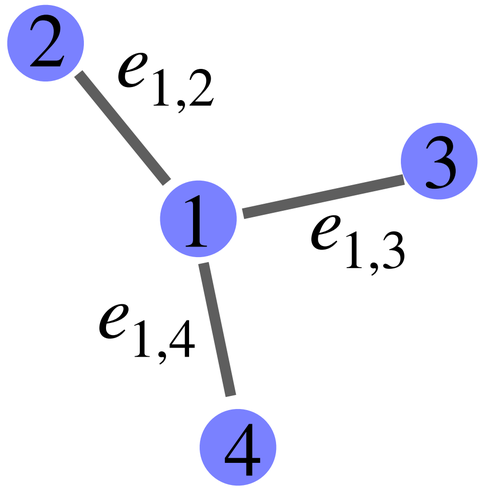
$1 \times N_{NodeFeatures}$

For all nodes

Node Feature

X

$N_{Node} \times N_{NodeFeatures}$



For one edge

Edge Feature

$1 \times N_{EdgeFeatures}$

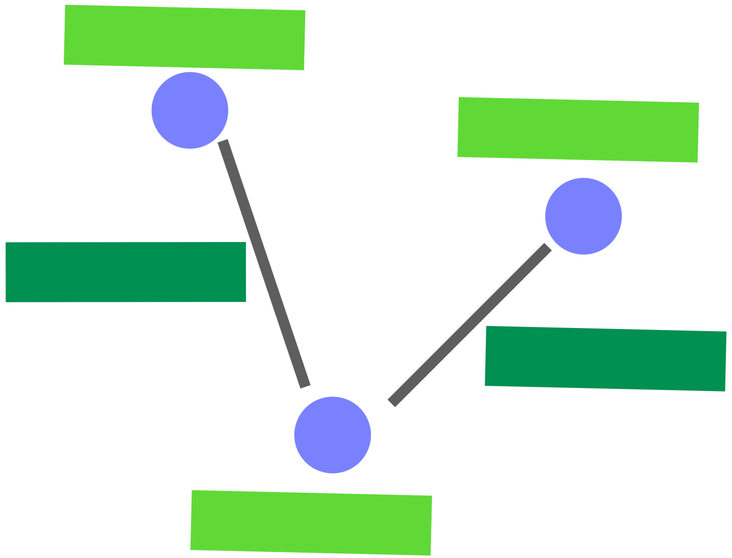
For all edges

Edge Feature

$N_{Edge} \times N_{EdgeFeatures}$

Graphs: How we specify graphs mathematically

Putting both topology (connections) and edge and node feature together:



Nodes

$N_{Node} \times 1$

Node features

$N_{Node} \times N_{NodeFeatures}$

Edges

Or

$2 \times N_{Edges}$

$N_{Node} \times N_{Node}$

Edge features

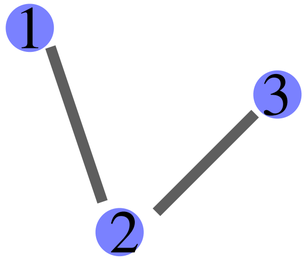
$N_{Edge} \times N_{EdgeFeatures}$

Graphs: node order is arbitrary

Relabeling or reordering nodes changes our matrices, but not the underlying graph.

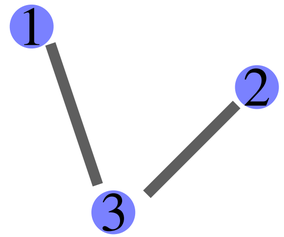
How can we change the order of nodes?

Permutation: A permutation $P_{i \leftrightarrow j}$ swaps the labels of nodes **i** and **j,**



$$A = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

$$P_{2 \leftrightarrow 3} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$



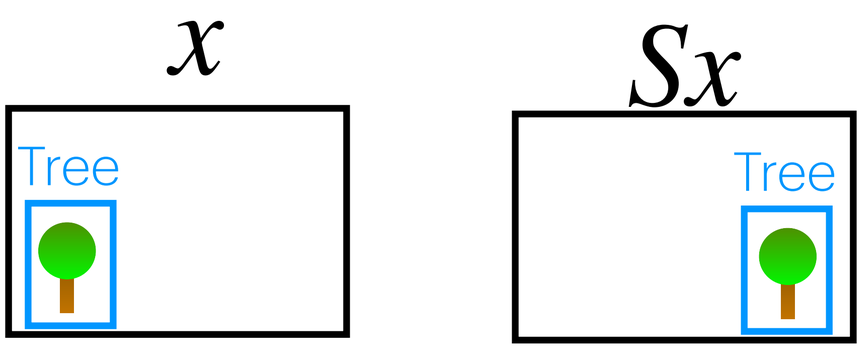
$$A' = PAP^T = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}$$

**Note that when we perform permutation, we should also apply it to node and edge features as well, since the order will be different.

Remember from our CNN lecture, that in grid-domain (e.g. images), the shift operation did not change the fact that a tree is a tree.

Convolutional layers are shift **equivariant**: shifting the input shifts the feature map by the same amount:

$f(x)$ $\quad$ $f(Sx) = Sf(x)$



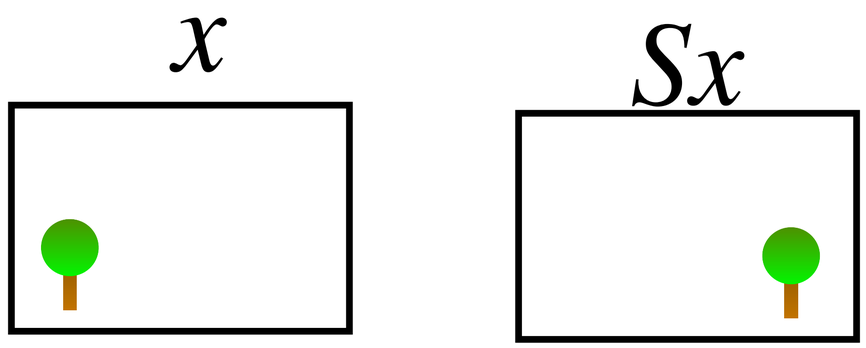
f is equivariant under S

$$f(Sx) = Sf(x)$$

Remember from our CNN lecture, that in grid-domain (e.g. images), the shift operation did not change the fact that a tree is a tree.

Now imagine a function g that takes an entire image and returns one answer: (for example, is there a tree in this image?)

$g(x)$ $\quad$ $g(Sx) = g(x)$

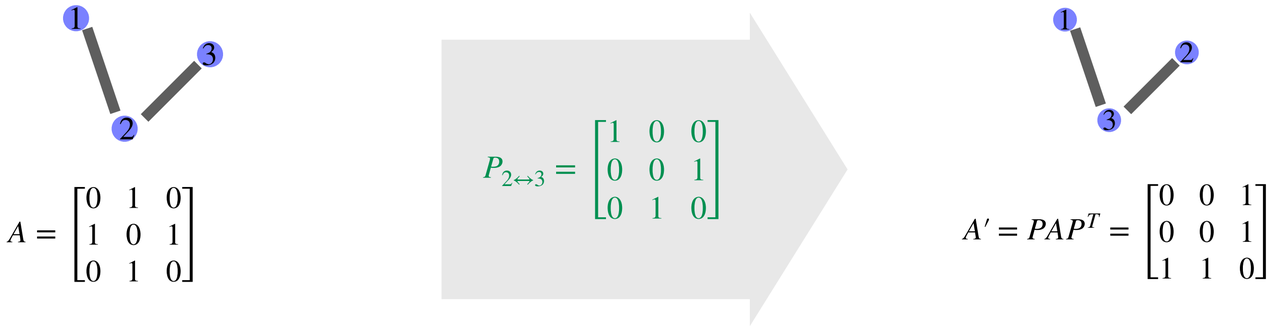


g is invariant under S

$$g(Sx) = Sg(x)$$

Graphs: node order is arbitrary

Permutation: A permutation $P_{i \leftrightarrow j}$ swaps the labels of nodes **i** and **j**



For graphs, the analogous symmetry is relabeling or reordering the nodes, called **Permutation**.

Equivariant: $f(PX, PAP^T) = P f(X, A)$ node outputs are reordered in the same way.

f is function that outputs one value for each node node

Invariant: $g(PX, PAP^T) = g(X, A)$ one graph-level output stays unchanged.

g is function that outputs one value per graph

Graphs: node order is arbitrary

Equivariant: $f(PX, PAP^T) = Pf(X, A)$ node outputs are reordered in the same way.

f is function that outputs one value for each node node

Example of f: Each node represents a coarse-grained chemical group. The function f predicts partial charge for each node.

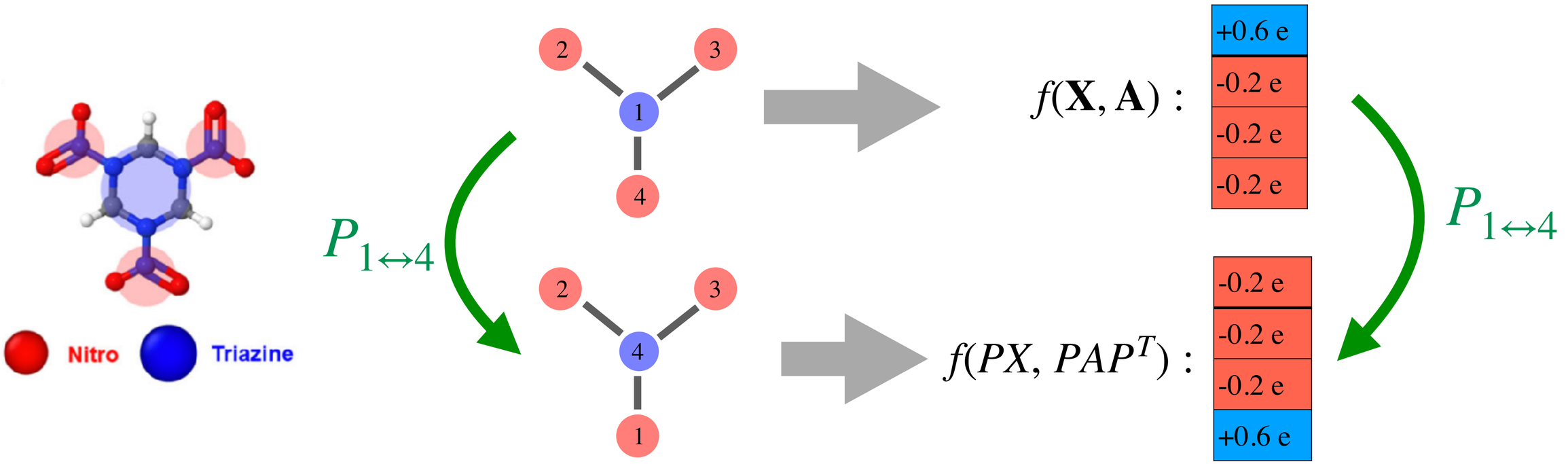


Image From: Lee, Brian H., et al. "Graph neural network coarse-grain force field for the molecular crystal RDX." *npj Computational Materials* (2024) - *charges and values are fictional

Graphs: node order is arbitrary

Invariant: $g(PX,\ PAP^T) = g(X, A)$ one graph-level output stays unchanged.

g is function that outputs one value per graph

Example of g: Each node represents a coarse-grained chemical group. The function g predicts the total energy of the whole molecular graph.

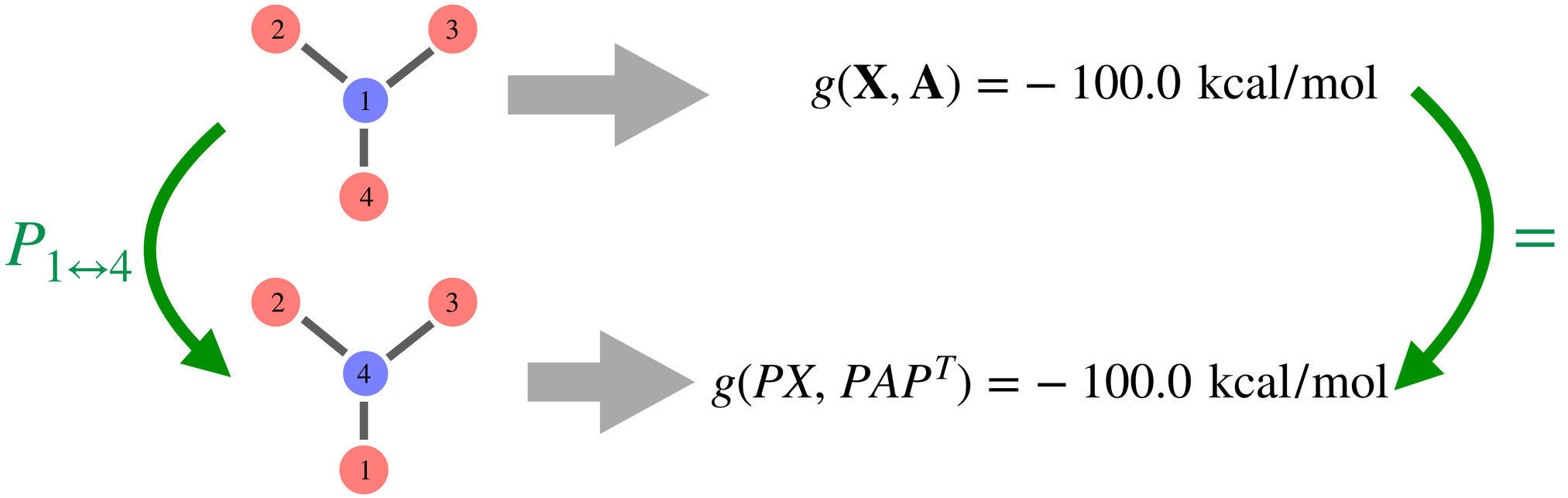
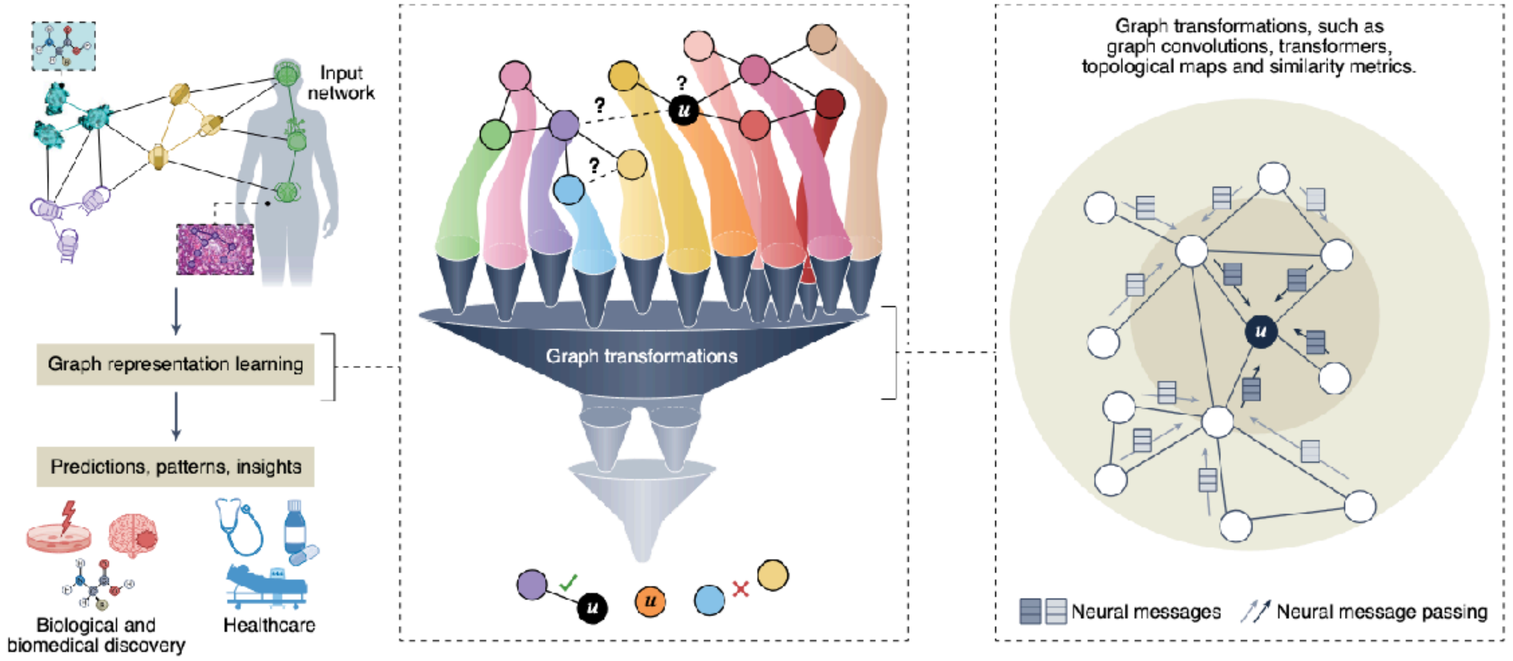


Image From: Lee, Brian H., et al. "Graph neural network coarse-grain force field for the molecular crystal RDX." *npj Computational Materials* (2024) - *charges and values are fictional

Part II: Graphs Neural Networks

GNNs: neural networks for graph-structured data

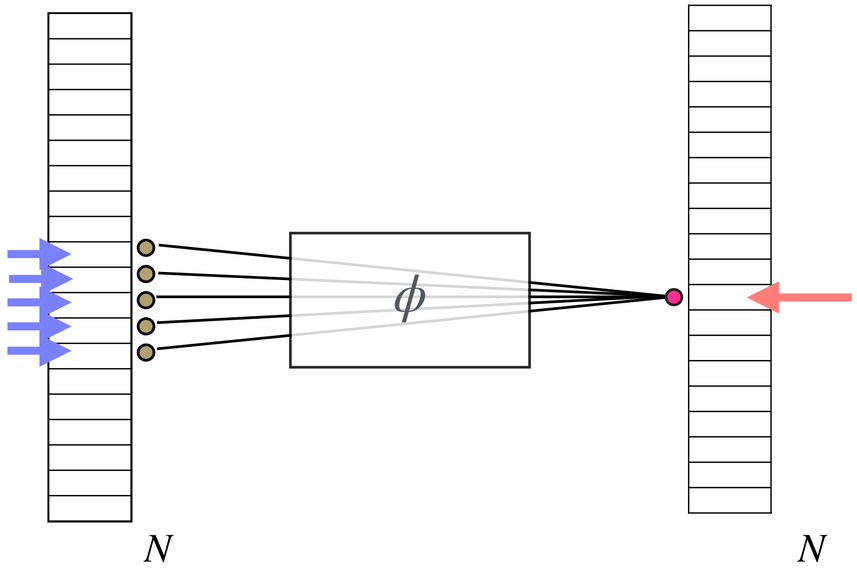
Input: nodes, edges, and their features
Output: predictions for nodes or for the whole graph
Key idea: the architecture should respect graph structure and graph symmetries.



Li, Michelle M., Kexin Huang, and Marinka Zitnik. "Graph representation learning in biomedicine and healthcare." *Nature biomedical engineering* (2022):

GNNs: intuition from CNNs

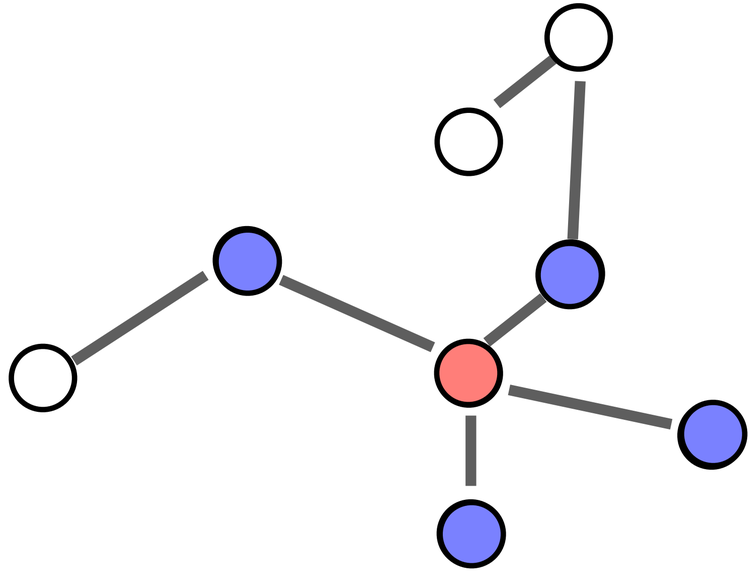
CNN: In Convolutional Neural Networks



We look at **elements near it**

when we want to fill **this element**

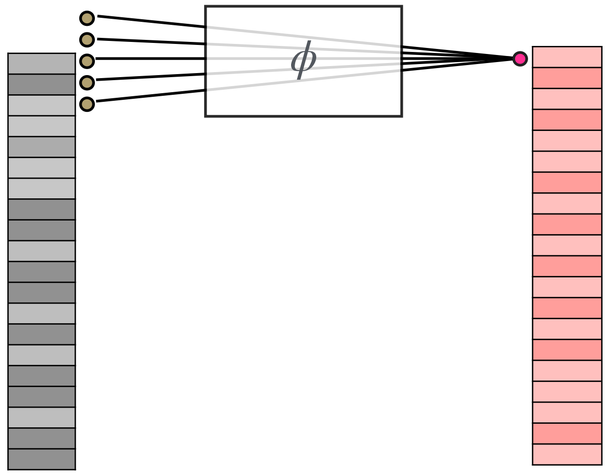
GNNs follows a similar idea, of locality

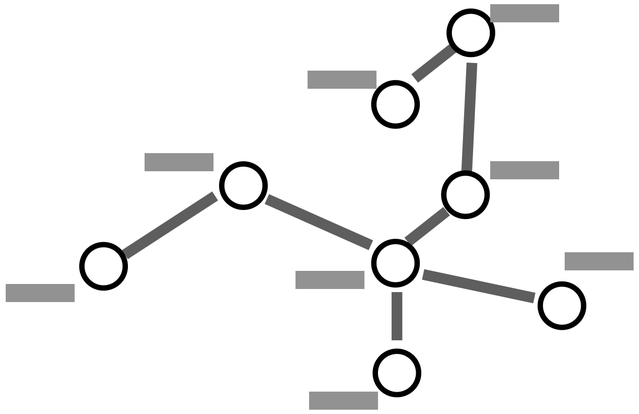


when we want to fill **this element**

We look at **elements connected to it**

GNNs: intuition from CNNs





In CNNs each pass is:

1. Each pixel has a **feature value.**

2. Convolution centred at that pixel
Combine information from a fixed local window.

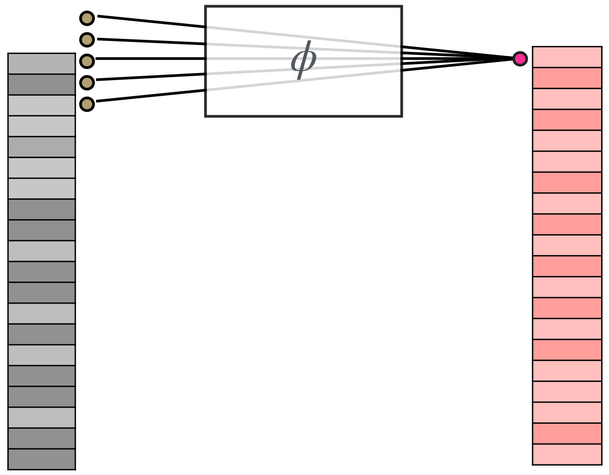
3. Then update the value of the pixel (in the centre) accordingly

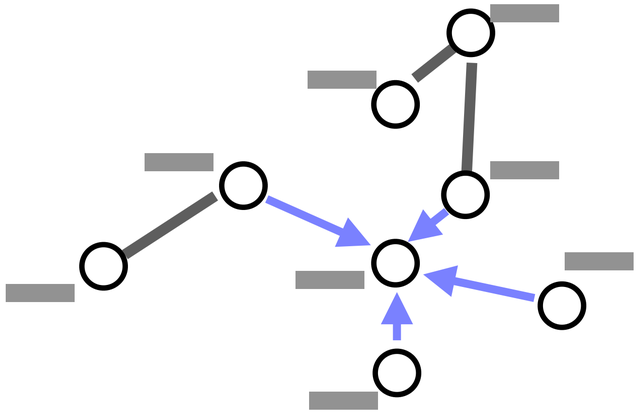
4. Update every pixel using the same rule.

In GNNs we have message passing:

1. Each node has a embedding **(feature) vector**.

GNNs: intuition from CNNs





In CNNs each pass is:

1. Each pixel has a **feature value.**

2. Convolution centred at that pixel
Combine information from a fixed local window.

3. Then update the value of the pixel (in the centre) accordingly

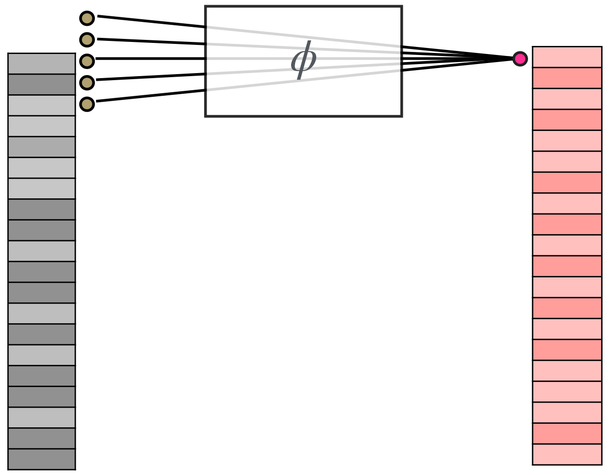
4. Update every pixel using the same rule.

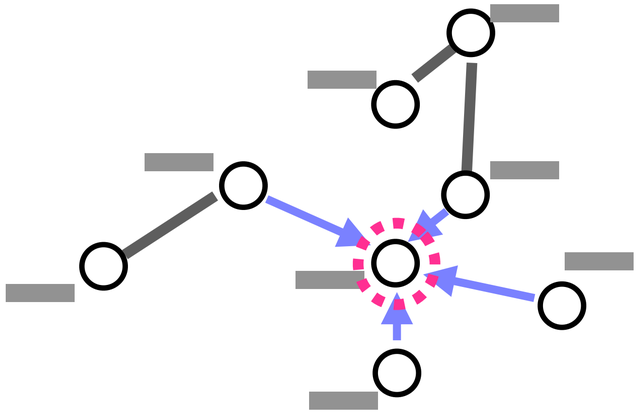
In GNNs we have message passing:

1. Each node has a embedding **(feature) vector**.

2. Calculate the message that each neighbouring node sends to the target node.

GNNs: intuition from CNNs





In CNNs each pass is:

1. Each pixel has a **feature value.**

2. Convolution centred at that pixel
Combine information from a fixed local window.

3. Then update the value of the pixel (in the centre) accordingly

4. Update every pixel using the same rule.

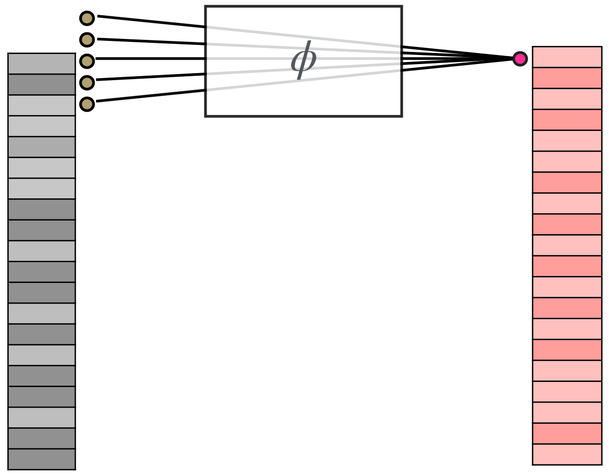
In GNNs we have message passing:

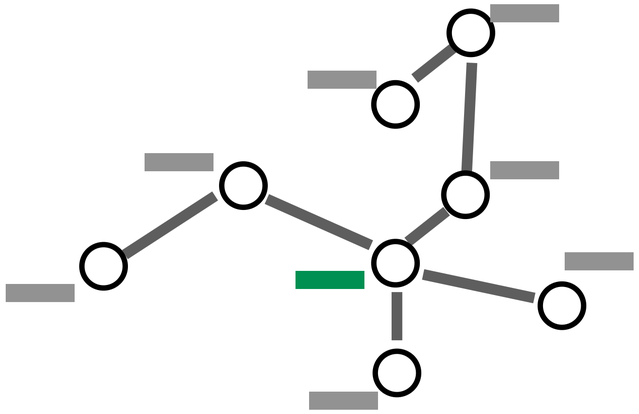
1. Each node has a embedding **(feature) vector**.

2. Calculate the message that each neighbouring node sends to the target node.

3-**Aggregate** all the messages at the target node

GNNs: intuition from CNNs





In CNNs each pass is:

1. Each pixel has a **feature value.**

2. Convolution centred at that pixel
Combine information from a fixed local window.

3. Then update the value of the pixel (in the centre) accordingly

4. Update every pixel using the same rule.

In GNNs we have message passing:

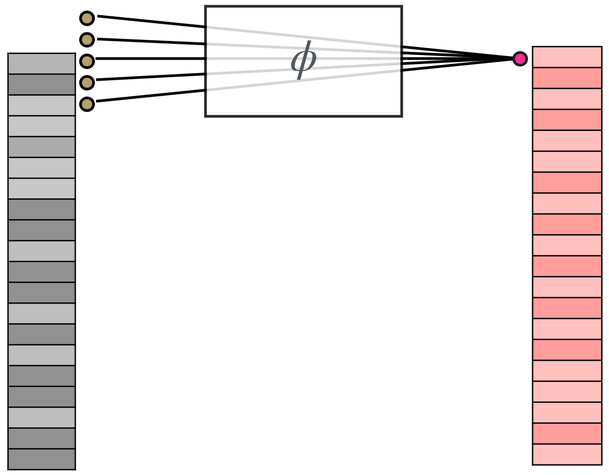
1. Each node has a embedding **(feature) vector**.

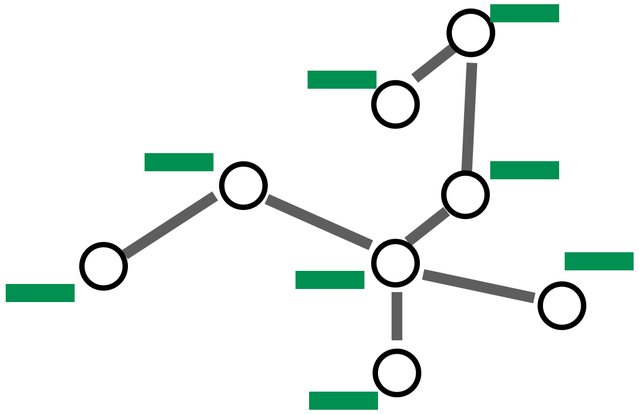
2. Calculate the message that each neighbouring node sends to the target node.

3-Aggregate all the messages at the target node

4.Then **update the embedding (feature)** of the target node.

GNNs: intuition from CNNs





In CNNs each pass is:

1. Each pixel has a **feature value.**

2. Convolution centred at that pixel
Combine information from a fixed local window.

3. Then update the value of the pixel (in the centre) accordingly

4. Update every pixel using the same rule.

In GNNs we have message passing:

1. Each node has a embedding **(feature) vector**.

2. Calculate the message that each neighbouring node sends to the target node.

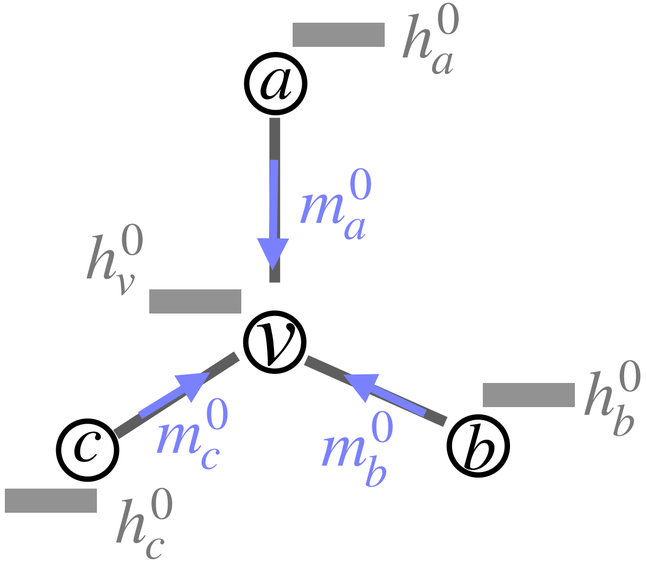
3-Aggregate all the messages at the target node

4.Then update the embedding (feature) of the target node.

5. Iterate over all nodes

GNNs: Formalizing One round of Message passing

In the first round of message passing:

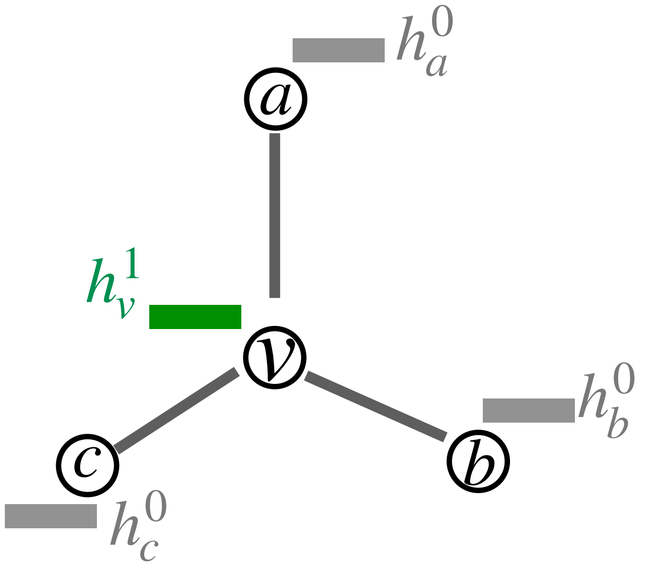


For each node v, that has neighbours N(v):
1-Calculate the **individual messages** from all neighbours individually.

$$m_{u\to v}^1 = \text{Message}(h_u^0, h_v^0) \qquad \forall u \in N(v)$$

2-**Aggregate** the messages of all neighbours

$$m_{N(v)}^1 = \text{AGG}^0\Big(\{m_{u\to v}^0 : u \in N(v)\}\Big)$$

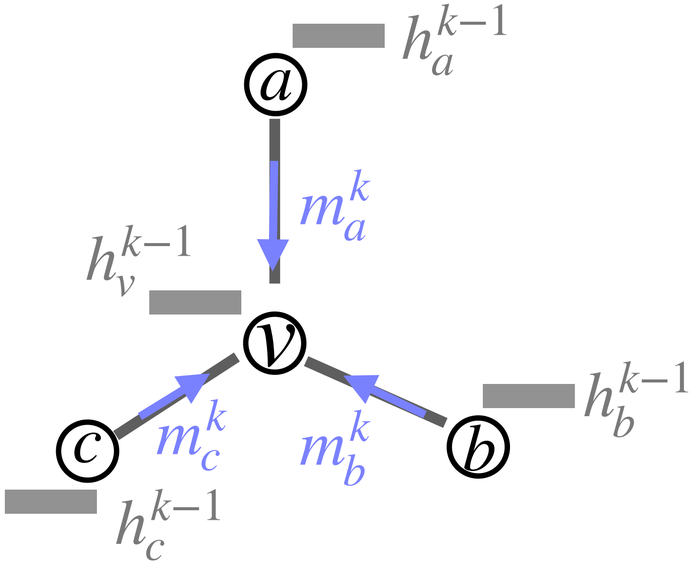


3- **Update** the embedding of the target node based on the aggregated messages and its own embedding

$$h_v^1 = \text{Update}\Big(h_v^0, m_{N(v)}^1\Big)$$

GNNs: Formalizing One round of Message passing

In each round (**k**) of message passing:

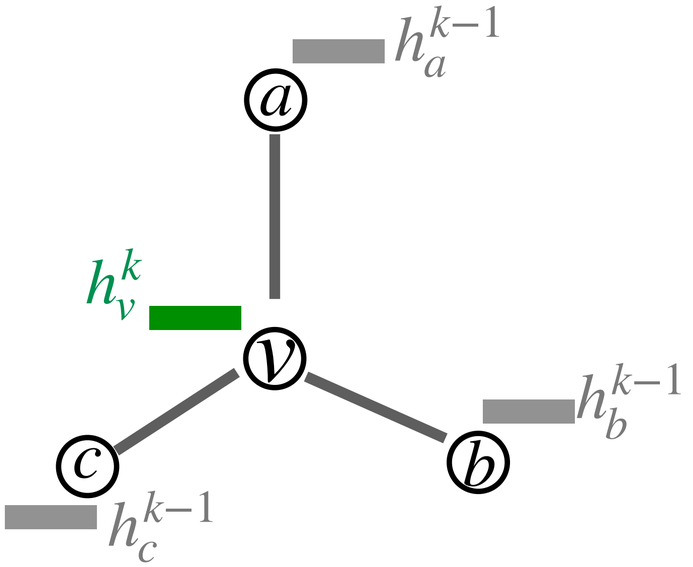


For each node v, that has neighbours N(v):
1-Calculate the **individual messages** from all neighbours individually.

$$m_{u\to v}^k = \text{Message}(h_u^{k-1}, h_v^{k-1}) \qquad \forall u \in N(v)$$

2-**Aggregate** the messages of all neighbours

$$m_{N(v)}^k = \text{AGG}^k\left(\left\{m_{u\to v}^k : u \in N(v)\right\}\right)$$

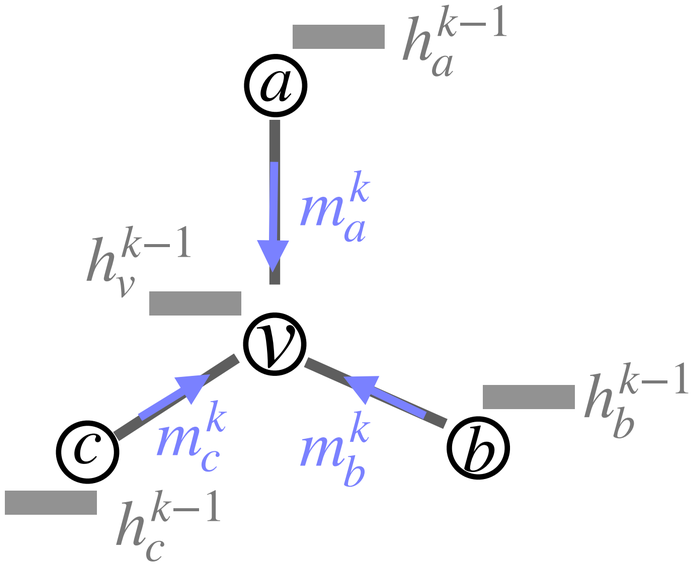


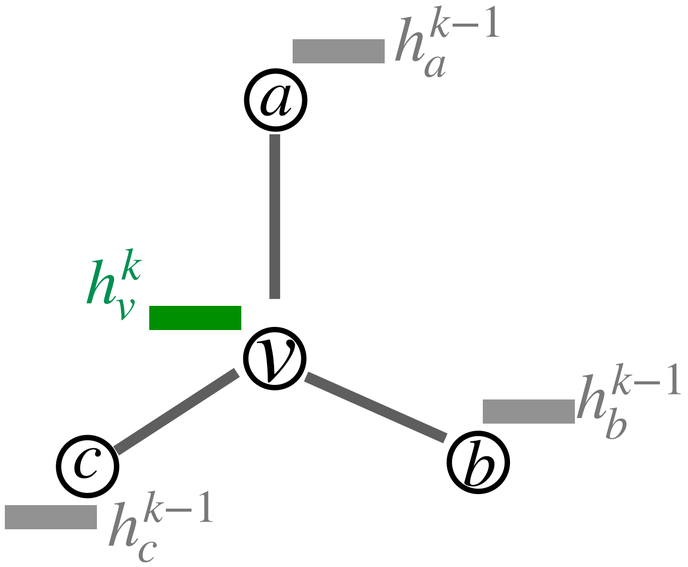
3- **Update** the embedding of the target node based on the aggregated messages and its own embedding

$$h_v^k = \text{Update}\left(h_v^{k-1}, m_{N(v)}^k\right)$$

GNNs: Formalizing One round of Message passing

In each round (**k**) of message passing:





In summary, for every target node v, a GNN applies three steps:

$$m_{u\to v}^k = \text{Message}(h_u^{k-1}, h_v^{k-1}) \qquad \forall u \in N(v)$$

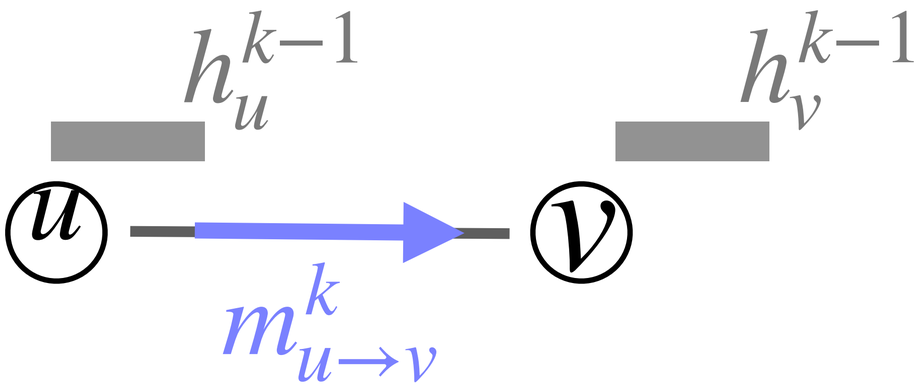
$$m_{N(v)}^k = \text{AGG}^k\left(\left\{m_{u\to v}^k : u \in N(v)\right\}\right)$$

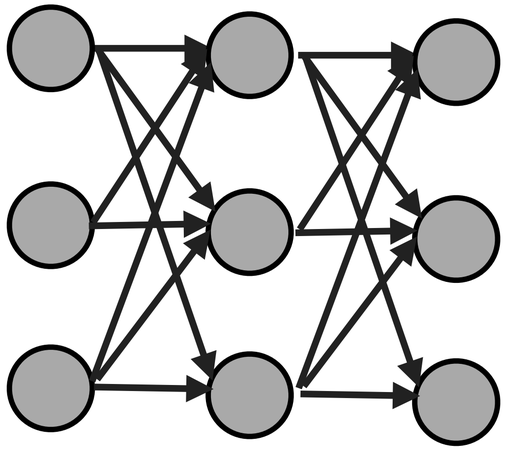
$$h_v^k = \text{Update}\left(h_v^{k-1}, m_{N(v)}^k\right)$$

Each of these function could be a neural networks as we will see.

GNNs: Message function

A message can be computed by a shared MLP. Shared because (usually) The same MLP is applied to every directed edge $u \rightarrow v$

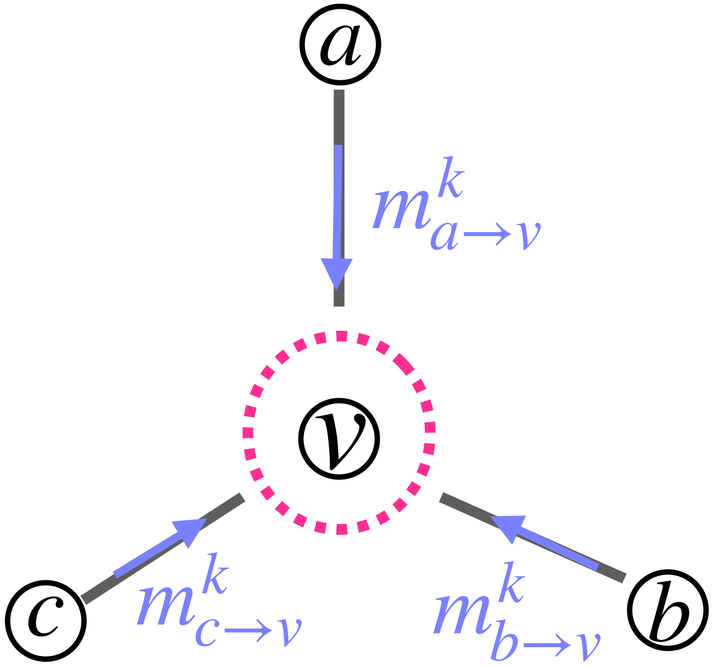




$$m_{u \rightarrow v}^k = \text{Message}(h_u^{k-1}, h_v^{k-1}) \qquad \forall u \in N(v)$$

GNNs: Aggregation function

- Aggregate the messages sent by all neighbours $u \in N(v)$ to node v.
- At round k, this produces one neighbourhood message for node v.
- Aggregation must be **permutation invariant**: the order of neighbouring nodes must not affect the result.



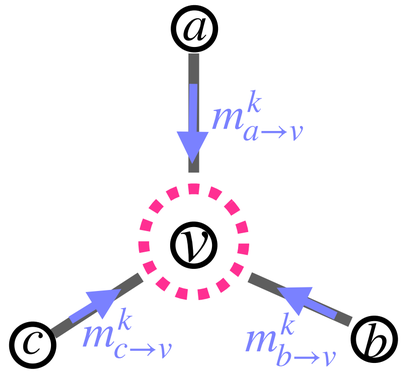
$$m^k_{N(v)} = \mathrm{AGG}^k\left(\left\{m^k_{u\to v} : u \in N(v)\right\}\right)$$

$$\mathrm{AGG}^k\left(m^k_{a\to v}, m^k_{b\to v}, m^k_{c\to v}\right) = \mathrm{AGG}^k\left(m^k_{c\to v}, m^k_{a\to v}, m^k_{b\to v}\right)$$

GNNs: Aggregation function

- Some possible options for **permutation invariant Aggregation**:

$$m_{N(v)}^k = \mathrm{AGG}^k\left(\left\{m_{u\to v}^k : u \in N(v)\right\}\right)$$



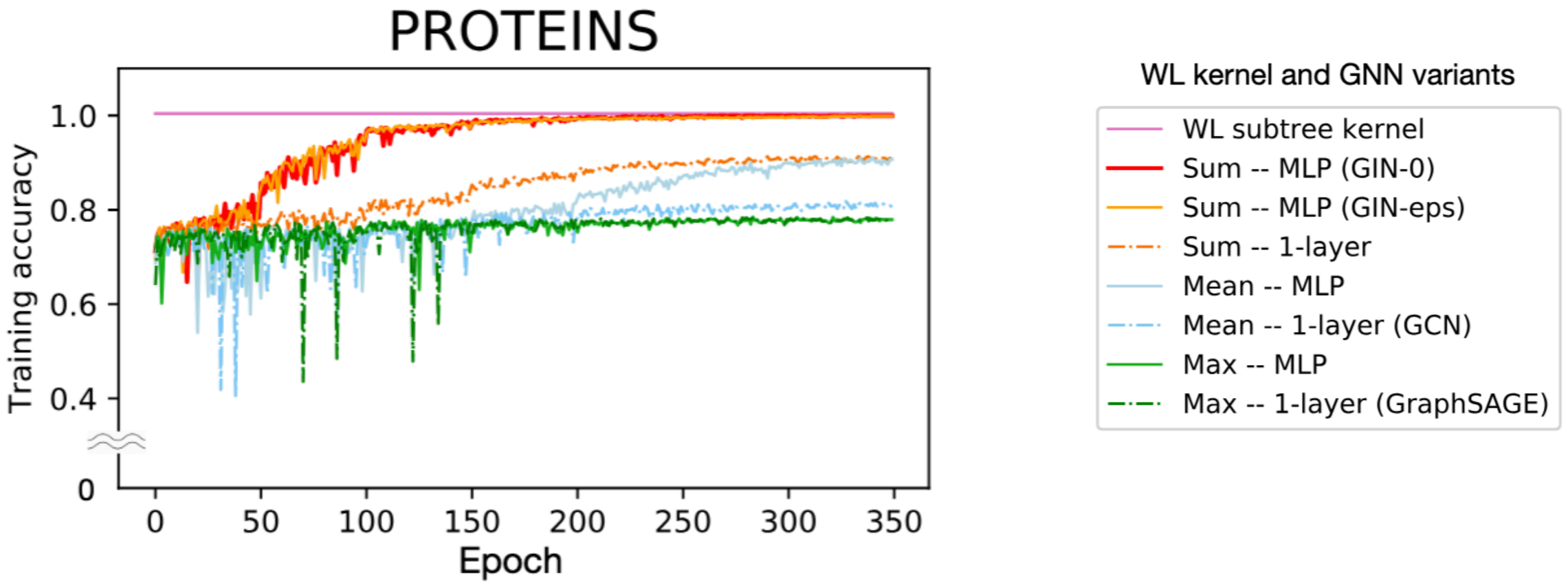
Sum: $$m_{N(v)}^k = \sum_{u\in N(v)} m_{u\to v}^k$$

Mean: $$m_{N(v)}^k = \frac{1}{|N(v)|}\sum_{u\in N(v)} m_{u\to v}^k$$

Min/Max: $$m_{N(v)}^k = \max_{u\in N(v)} m_{u\to v}^k$$

Deep Sets aggregation: $m_{N(v)}^k = \rho^k\left(\sum_{u\in N(v)} \phi^k\left(m_{u\to v}^k\right)\right)$

GNNs: Invariant Aggregation function in practice



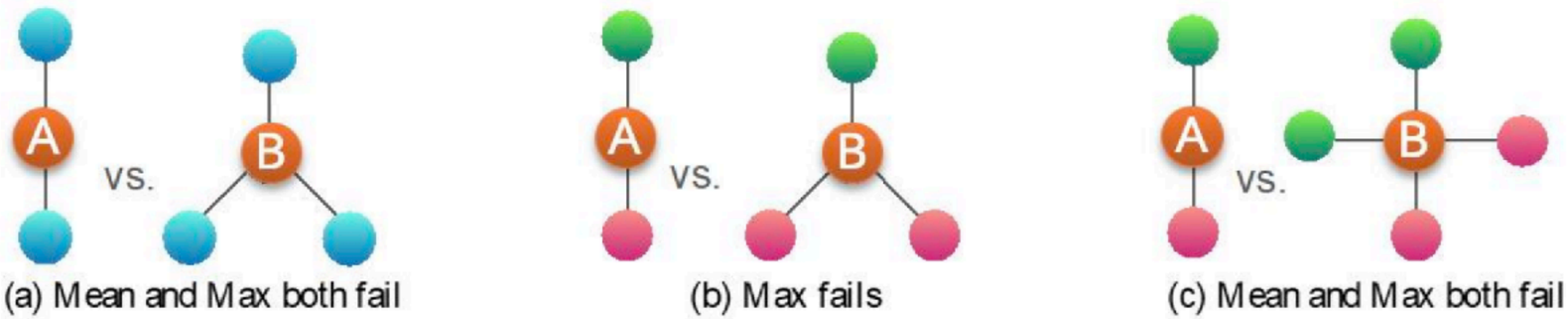
Xu, Keyulu, et al. "How powerful are graph neural networks?." *arXiv preprint arXiv:1810.00826* (2018).

GNNs: Aggregation function

- In terms of they discriminative power, we can rank Aggregation functions as bellow:

Deepsets >> Sum > Mean > Min/Max

- Mean and Max/Min pooling are not able to distinguish many cases, e.g*:

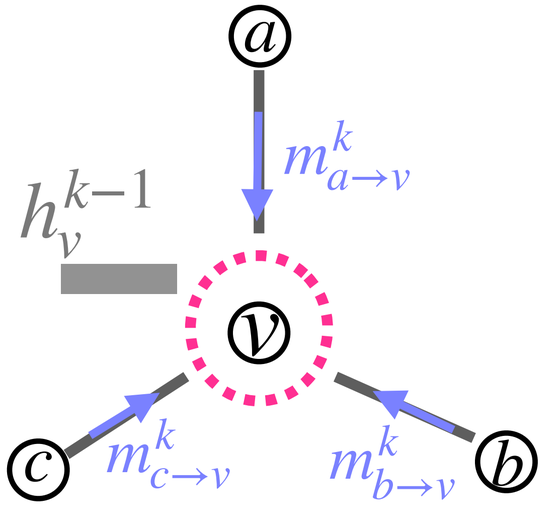


(a) Mean and Max both fail (b) Max fails (c) Mean and Max both fail

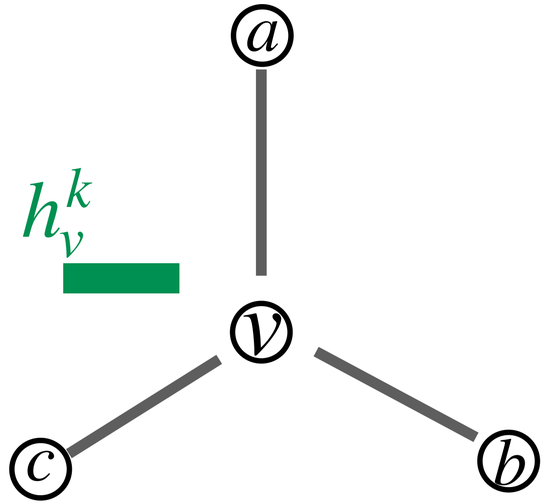
*From Jure Leskovec, Stanford University

GNNs: Update function

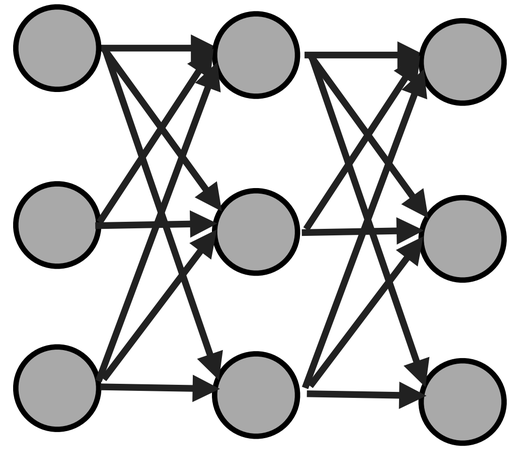
- At round k, combine node v's previous embedding with its aggregated neighbourhood message.



$$h_v^k = \text{Update}^k\left(h_v^{k-1}, m_{N(v)}^k\right)$$

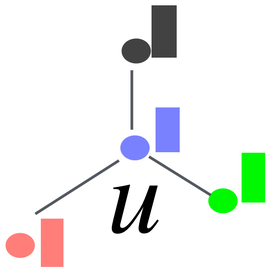


A common choice is a simple linear update or shared MLP. shared because (usually) The same update MLP is applied to every node:

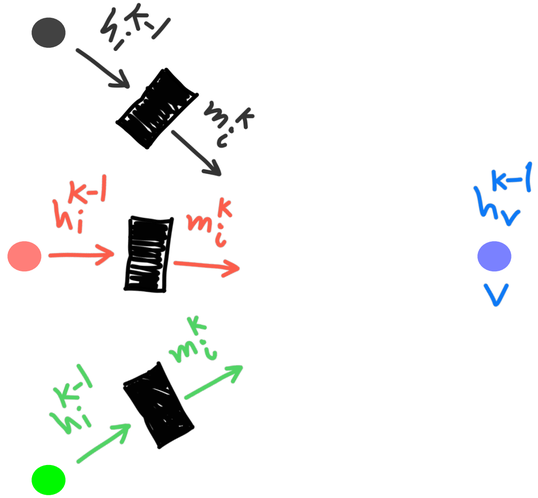


GNNs: Putting Message transformation, Aggregation and update together

- Putting all together, one round of message passing can be visualized as bellow:

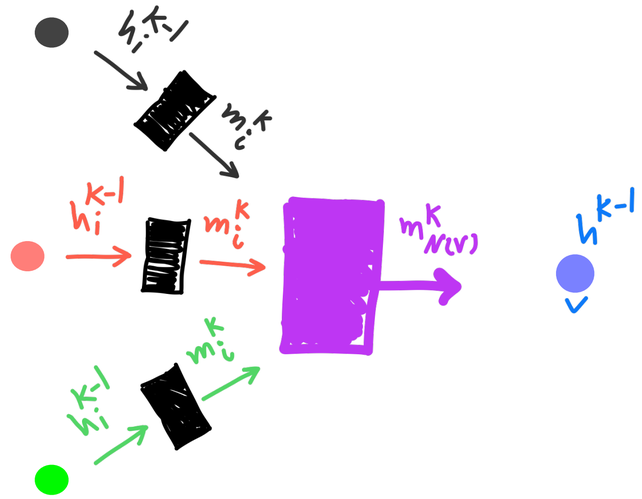


1.Message Transformation



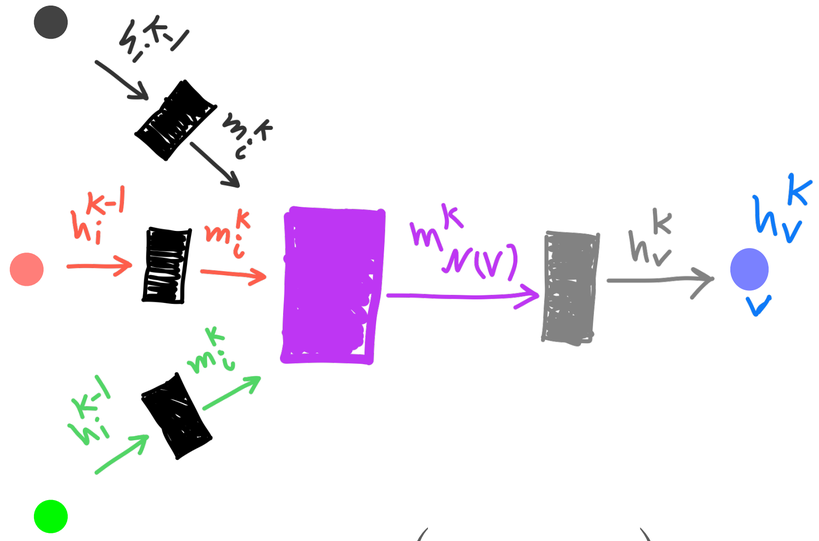
$$m_u^k = \text{Message}\left(h_u^{k-1}\right), \quad \forall u \in N(v)$$

2.Aggregation



$$m_{N(v)}^k = \text{AGG}^k\left(\left\{m_u^k : u \in N(v)\right\}\right)$$

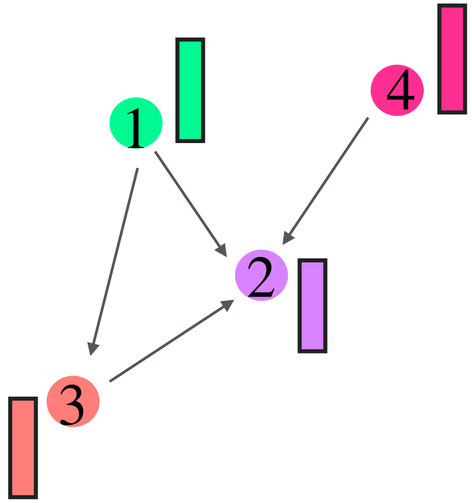
3.Update



$$h_v^k = \text{Update}\left(h_v^{k-1}, m_{N(v)}^k\right)$$

GNNs: computation graph of message passing

- The computation graph shows which earlier node embeddings are needed to compute a later embedding.



- Node embeddings are vectors.
- Messages are computed along graph edges.
- The same Message and Update functions are shared across all edges and nodes.
- Stacking layers lets each node use information from farther-away nodes.

h_v^{k+1}

...

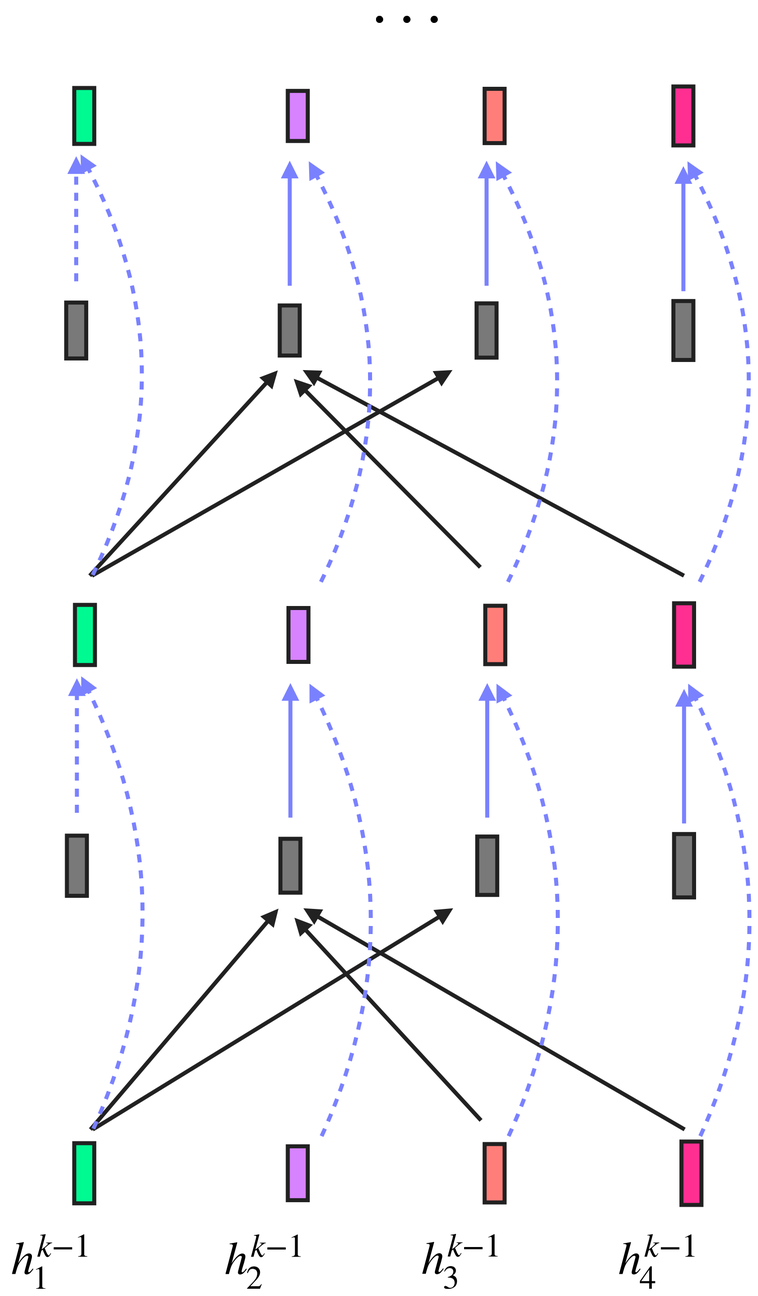
h_v^k

$h_v^k = UPDATE^k(m_{N(v)}^k, h_v^{k-1})$

$m_{N(v)}^k = AGG^k\{m_u^{k-1}; u \in N(v)\}$

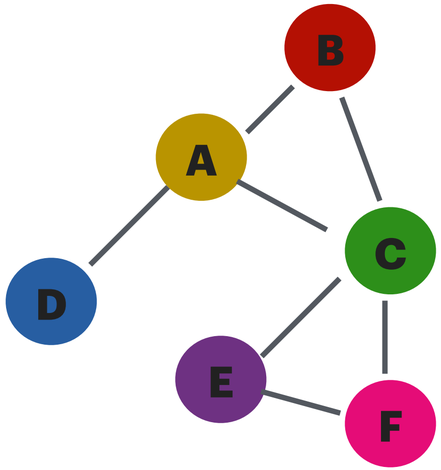
$m_u^k = Message^k(h_u^{k-1})$

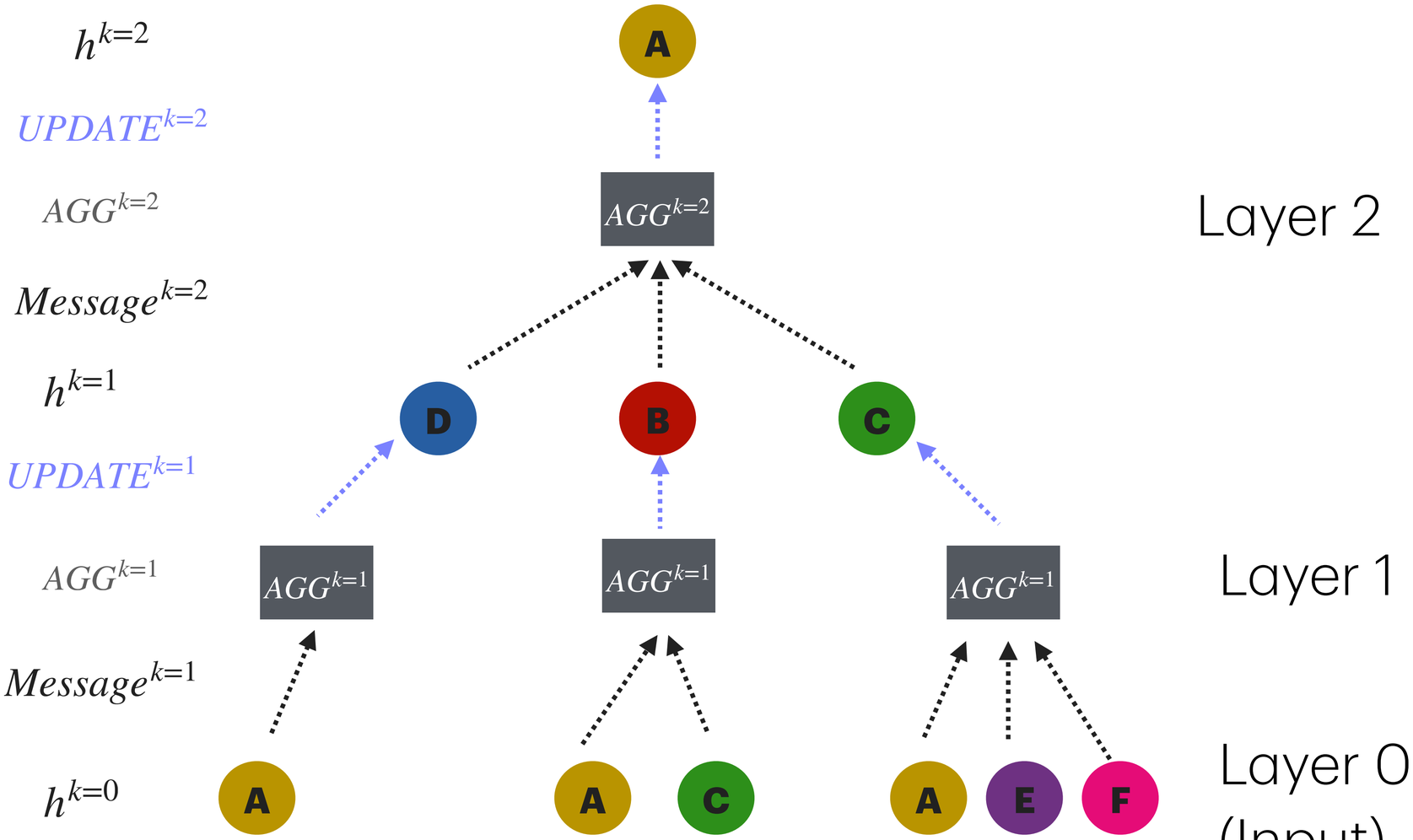
h_v^{k-1}



GNNs: the computation graph of GNNs: Tree view

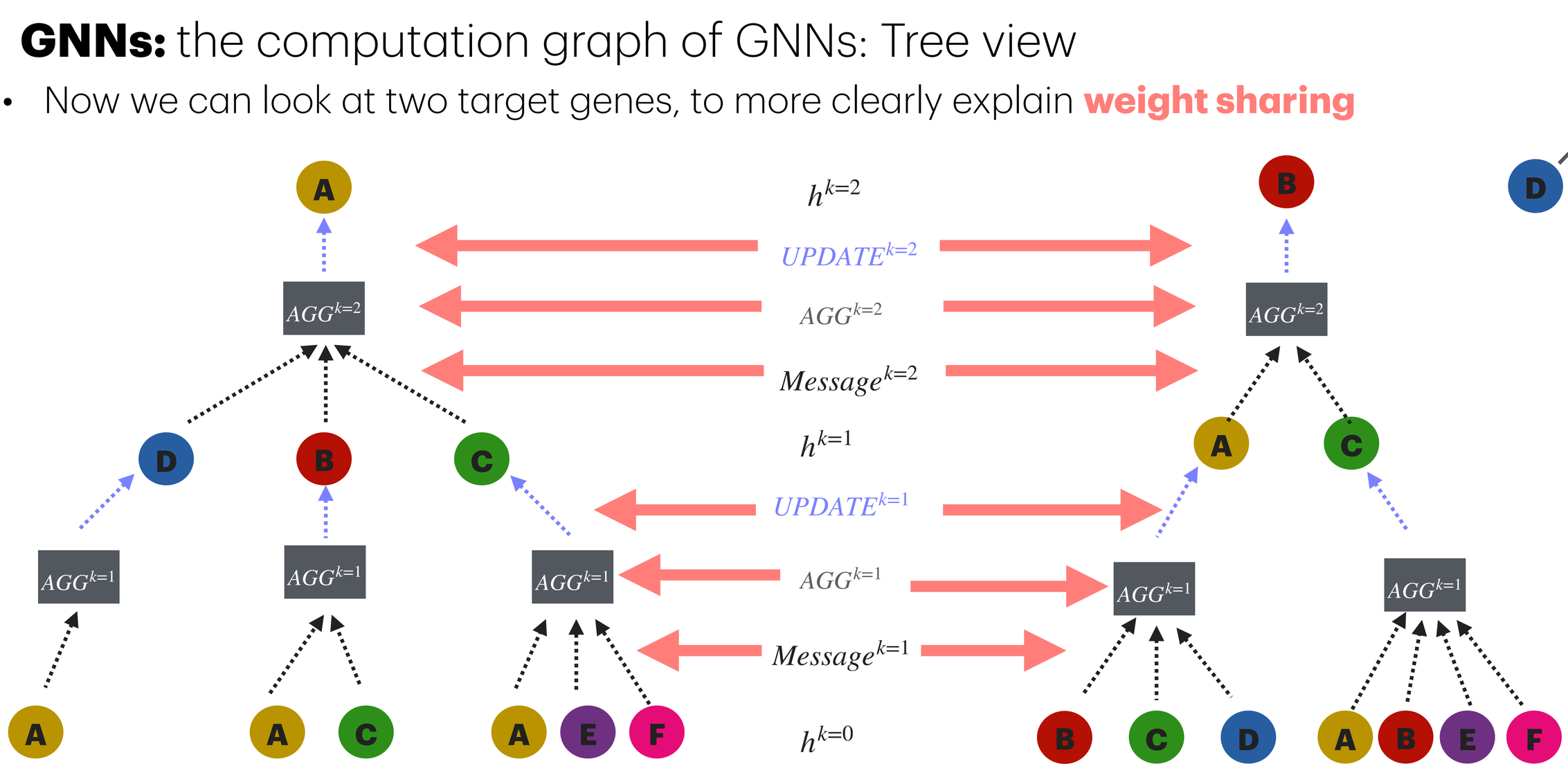
- Another ways of looking at GNNs is how information flows inside the graph as the steps ‘k’ unroll.
- To understand this more clearly, we can draw the computation graph as a tree for one target node (**A**) at layer 2 (**k=2**).





GNNs: the computation graph of GNNs: Tree view

- Now we can look at two target genes, to more clearly explain **weight sharing**



- This is an important property, GNNs can be **inductive:** "learn a rule that can be applied to new graphs too". Because the weights that are learned, are shared between all nodes. Now if we have a graph with different nodes or edges (of course same type of data), the learned GNN can work on it. So we can train on a part of graph and predict the rest etc.

GNNs: Summary

- Encodes graph structure together with node and edge features
- Important property: permutation invariance / equivariance
- Core idea: message passing and aggregation
- Can handle graphs with different sizes and structures, similar to how CNNs handle images
- Connected to graph signal processing, graphical models, distributed computing, and graph isomorphism testing
- Representation can be improved with higher-order structure, node IDs, and augmentation

https://visual-intelligence-umn.github.io/GNN-101/

Bonus: **GNNs:** the computation graph of GNNs:

Important note on learning settings: One can choose that the computational functions of GNNs which are $Message^k$,AGG^k, and $UPDATE^k$ to share parameters across the depth (k values) or not. But:

A. If we allow each k has it's own parameters, the computation becomes more powerful, with the cost of more parameters and potential overfitting

B. If we share parameters across depth ($Message = Message^k$,$AGG = AGG^k$, and $UPDATE = UPDATE^k$), we should make sure that the model converges after iterations of depth.

Edge attribute: One can also assign edge feature if needed, it will change the form of message fucntion only

Hand wavy rule of thumbs: If we have more depth (K), we can get away with less parameter in each layer, but if we have a shallow GNN (e.g K=2), we need more powerful functions

How decide the depth value: look at the receptive field of your problem, and adjust the k accordingly.

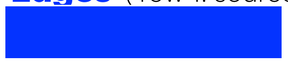
Bonus: Coding the Most General GNN model in PyG:

A) Preprocessing inputs to GNN algorithms

NodesFeatures

$N_{node} \times N_{NodeFeatures}$

Edges (row 1: source row 2 target)



$2 \times N_{Edge}$

EdgeFeatures

$N_{Edge} \times N_{EdgeFeatures}$

B) PytorchGeometric makes these internally for message passing:

x_j (Source Node Features On Edges)

$N_{Edge} \times N_{NodeFeatures}$

x_i (Target Node Features On Edges)

$N_{Edge} \times N_{NodeFeatures}$

C)propagation

$Message$ (**x_i**,**x_j**, **EdgeFeatures**)

Messages

$N_{Edge} \times N_{MessageFeatures}$

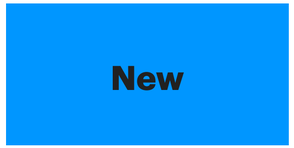
AGG(**Messages, Edges[1,:]**)

Agg_Messages

$N_{Node} \times N_{MessageFeatures}$

$UPDATE_{edge}$(**x_i**,**x_j**, **EdgeFeatures**)

EdgeFeatures



$N_{Edge} \times N_{EdgeFeatures}$

$UPDATE$(**NodesFeatures**, **Agg_Messages**)

NodesFeatures

$N_{node} \times N_{NodeFeatures}$

D) Post processing

Global State (**NodesFeatures, Edges, EdgeFeatures**)

Global State

$N_{GraphFeatures}$

You should only specify:

Tensors:
NodesFeatures
Edges
EdgeFeatures (Optional)

Functions:
$Message$
AGG
$UPDATE_{edge}$(Optional)
$UPDATE$
$GlobalState$(Optional)

Bonus: Coding the Most General GNN model in PyG: Example Flavours

You should only specify:

Tensors:
NodesFeatures
Edges
EdgeFeatures (Optional)

Functions:

$$m_{u\to v}^k = \text{Message}^k\left(h_v^{k-1}, h_u^{k-1}, e_{uv}\right)$$

$$m_{N(v)}^k = \text{AGG}^k\left(\{\, m_{u\to v}^k : u \in N(v)\,\}\right)$$

$$h_v^k = \text{Update}^k\left(h_v^{k-1}, m_{N(v)}^k\right)$$

Optional:
$GlobalState$(Optional)
$UPDATE_{edge}$(Optional)

Graph Convolutional Networks (GCN)

Key Idea: It's just a weighted sum of neighbors.

$$m_{u\to v}^k = \frac{1}{\sqrt{\hat{d}_u \hat{d}_v}} W^k h_u^{k-1}$$

$$m_{N(v)}^k = \sum_{u\in N(v)\cup\{v\}} m_{u\to v}^k$$

$$h_v^k = \sigma\left(m_{N(v)}^k\right)$$

Graph Attentional Network (GAT)

Key Idea: Learns an attention weight for each neighbor, so neighbors contribute with different importance.

$$m_{u\to v}^k = \alpha_{u\to v}^k(h_v^{k-1}, h_u^{k-1})\,\text{MLP}^k\left(h_u^{k-1}\right)$$

**Where $a^k(h_v^{k-1}, h_u^{k-1})$ is normalized attention of u to v*

$$m_{N(v)}^k = \sum_{u\in N(v)\cup\{v\}} m_{u\to v}^k$$

$$h_v^k = \sigma\left(m_{N(v)}^k\right)$$

* will learn more about attention later in the course

Message Passing Neural Network (MPNN)

Key Idea: Uses heavy-duty neural networks (MLPs or RNNs) for the functions instead of simple linear layers.

$$m_{u\to v}^k = \text{MLP}^k\left(h_v^{k-1}, h_u^{k-1}, e_{uv}\right)$$

$$m_{N(v)}^k = \sum_{u\in N(v)} m_{u\to v}^k$$

(Mean/Max/or LSTM)

$$h_v^k = \text{Update}^k\left(h_v^{k-1}, m_{N(v)}^k\right)$$

(Update can be GRU, MLP, or another learnable node-wise function)

Bonus: **Graph Convolutional Networks** (GCN) **Graph Attentional Network** (GAT) **Message Passing Neural Network** (MPNN)

$$m_{u\to v}^k = \frac{1}{\sqrt{\hat{d}_u \hat{d}_v}} W^k h_u^{k-1}$$

$$m_{N(v)}^k = \sum_{u \in N(v) \cup \{v\}} m_{u\to v}^k$$

$$h_v^k = \sigma\left(m_{N(v)}^k\right)$$

$$m_{u\to v}^k = \alpha_{u\to v}^k(h_v^{k-1}, h_u^{k-1})\,\mathrm{MLP}^k\left(h_u^{k-1}\right)$$

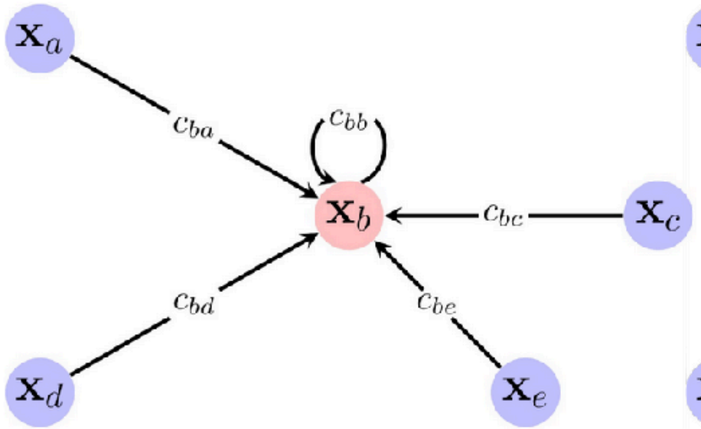
$$m_{N(v)}^k = \sum_{u \in N(v) \cup \{v\}} m_{u\to v}^k$$

$$h_v^k = \sigma\left(m_{N(v)}^k\right)$$

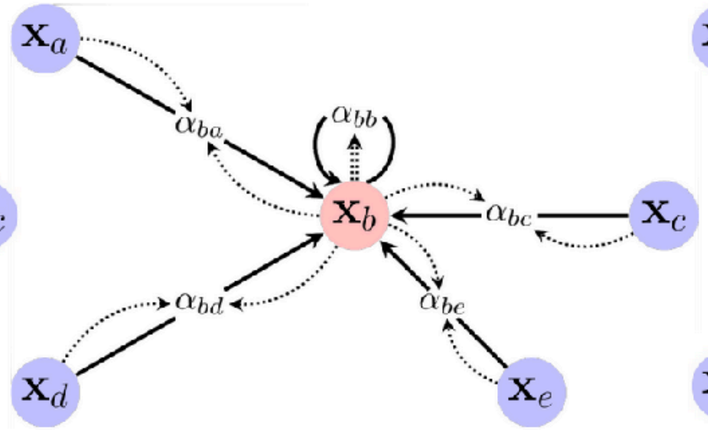
$$m_{u\to v}^k = \mathrm{MLP}^k\left(h_v^{k-1}, h_u^{k-1}, e_{uv}\right)$$

$$m_{N(v)}^k = \sum_{u \in N(v)} m_{u\to v}^k$$

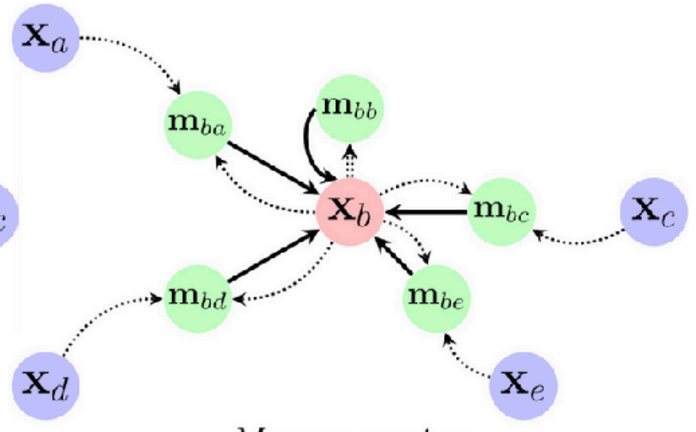
$$h_v^k = \mathrm{Update}^k\left(h_v^{k-1}, m_{N(v)}^k\right)$$



Convolutional



Attentional



Message-passing

$$\mathbf{h}_i = \phi\left(\mathbf{x}_i, \bigoplus_{j \in \mathcal{N}_i} c_{ij}\psi(\mathbf{x}_j)\right)$$

$$\mathbf{h}_i = \phi\left(\mathbf{x}_i, \bigoplus_{j \in \mathcal{N}_i} a(\mathbf{x}_i, \mathbf{x}_j)\psi(\mathbf{x}_j)\right)$$

$$\mathbf{h}_i = \phi\left(\mathbf{x}_i, \bigoplus_{j \in \mathcal{N}_i} \psi(\mathbf{x}_i, \mathbf{x}_j)\right)$$

Bronstein, Michael M., et al. "Geometric deep learning: Grids, groups, graphs, geodesics, and gauges." *arXiv preprint arXiv:2104.13478* (2021).

Recipe Card: **GCNs**

Type of Learning: Supervised
Data set: Graph: nodes, edges,node features

Data

1- Load libraries: Basics + PyTorch + PyG (pytorch_geometric)

2-Load your graph Dataset **D.** Sometimes D has only one graph (data):

-Make sure each element of $D = [data_1, \ldots data_K]$, $data_i$ is a PyG Data object:

-$data_i \,.\, x$ = NodeFeatures, shape: $[N_{node}, N_{nodeFeatures}]$

-$data_i \,.\, edge_index$= Edges, shape: $[2, N_{edge}]$

-$data_i \,.\, y$= target, shape for graph level: $[N_{target}]$, for node level: $[N_{node}, N_{target}]$

3-Now set up the DataLoader for training/validation/testing: (This depends on the task)

a) If you have many graphs:

-split the dataset into: train_data validate_data test_data.

-then load each split using PyG DataLoader. PyG will batch graphs automatically

b) If you have one big graph for node classification/regression

-keep one Data object then define train_mask val_mask test_mask.

-if these masks are not provided in the dataset, they can be made by RandomNodeSplit function in PyG.

NN model

4-Building the GCN neural network

-GCNs have two main parts. GCN layers and Task heads.

A)define GCNs layers as a module [GCN]: (x is node features and x' is updated node features)

$$Input: \ (x, edge\_index) \rightarrow [(GCNConv) \rightarrow (Relu)] \rightarrow \ldots \rightarrow [(GCNConv) \rightarrow (Relu)] \rightarrow x'$$

-B) define task head as a module [Task Head]:

-For node-level tasks (one per node, could be classification, regression, embedding, etc):

$$x' \rightarrow [Linear\ layer] \rightarrow out, \hat{y}_v$$

-For graph-level tasks (one per graph, could be classification, regression, embedding, etc):

$$x' \rightarrow [GlobalMeanPool] \rightarrow [Linear\ layer] \rightarrow out, \hat{Y}$$

Training

5-Define a proper loss function based on Y

a-If Y is continuous, then use MSELoss or L1Loss

b-If Y is categorical, use CrossEntropyLoss. note that we feed raw logits directly to Loss.

i-For node-level tasks, calculate loss on the selected nodes only (usually via train_mask)

ii-For graph-level tasks, calculate loss on the graph outputs $\hat{Y}$

-Add regularization to prevent overfitting

-The loss will be the sum of all relevant losses

6-Define the train and test functions:

-Train/Validate/test functions are standard

-For node-level tasks it masks for training/validation/testing.

-For graph-level tasks, the model gets batched graphs from PyG DataLoader

7-Define training and run the training:

-Same as standard.

-For each epoch: run train() on training data, then run test() on training/validation data

-At the end: load the best saved model and plot training/validation loss curves if needed

Recipe Card: **MPNN**

Type of Learning: Supervised
Data set: Graph: nodes, edges,node features

Data

1- Load libraries: Basics + PyTorch + PyG (pytorch_geometric)
2-Load your graph Dataset **D.** Sometimes D has only one graph (data)
-All similar to **GCNs**
3-Now set up the DataLoader for training/validation/testing: (This depends on the task)
-All similar to **GCNs**

Training

5-Define a proper loss function based on Y
-All similar to **GCNs**

6-Define the train and test functions:
-All similar to **GCNs**
7-Define training and run the training:
-All similar to **GCNs**

NN model

4-Building the MPNN neural network

-MPNNs have two main parts: Message Passing layers and Task Head.

A) define Message Passing layers as a module [MPNN]:

-In PyG, define each layer using MessagePassing.

-Inside one layer, forward() gets $(x, edge\_index, edge\_attr)$ as input then calls propagate(...).

-$propagate(\dots)$ starts message passing over all edges: it uses $message(\dots)$ to build edge-wise messages, $AGG(\dots)$ to combine incoming messages for each node, and $update(\dots)$ to produce the updated node features x' x '

-So Under $propagate(\dots)$, we define:

$$m_{u\to v}^{k} = Message^{k}(h_v^{k-1}, h_u^{k-1}, e_{uv}), \qquad m_{N(v)}^{k} = AGG^{k}(\{m_{u\to v}^{k} : u \in N(v)\}), \qquad h_v^{k} = Update^{k}(h_v^{k-1}, m_{N(v)}^{k})$$

-So overal a full [MPNN] has k layers (layer in sense of GNNs, which means k itterations) and [MPNN] module is like:

$$Input : (x, edge\_index, edge\_attr) \to [(MessagePassing) \to (Relu)] \to \dots \to [(MessagePassing) \to (Relu)] \to x'$$

-B) define task head as a module [Task Head]:

-all similar to GCN, once the x' is computed by [MPNN] module.